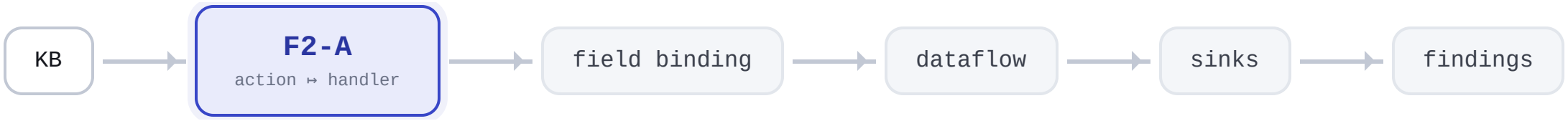
TECHNICAL DESIGN REVIEW

Dynamic dispatch hides the protocol handler

F2-A recovers the action → handler mapping, backed by an **auditable evidence calculus**.



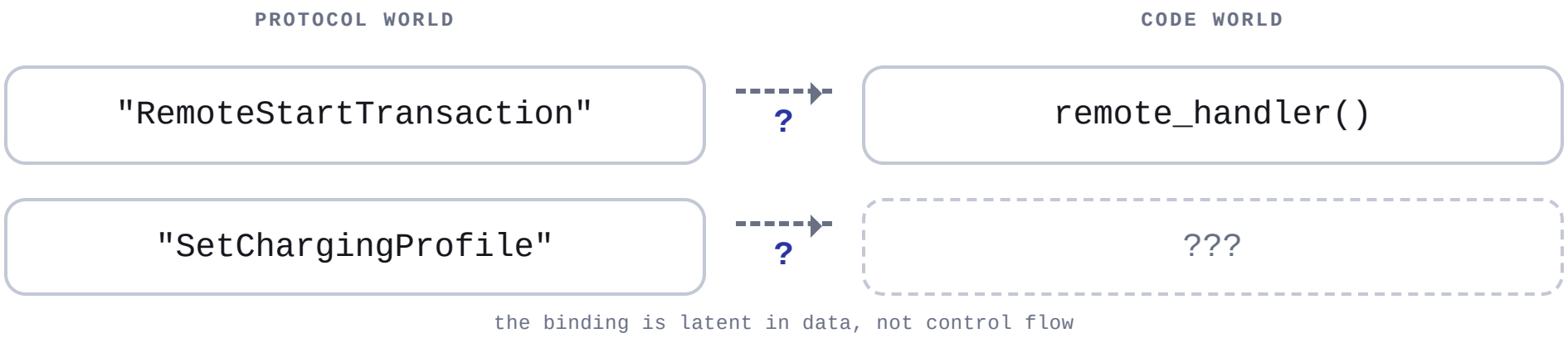
one stage of an OCPP-aware static pipeline

- F2-A is the **Handler Resolution** stage of an OCPP-aware static pipeline
- In: protocol knowledge + a code property graph · Out: per-action resolution + evidence trail
- Contribution: evidence model + calculus + explicit **“refuse to guess”** semantics

WHY HANDLER RESOLUTION EXISTS

Protocol semantics and code structure are disconnected

Spec-level actions and the code that enforces them are linked only by indirection.

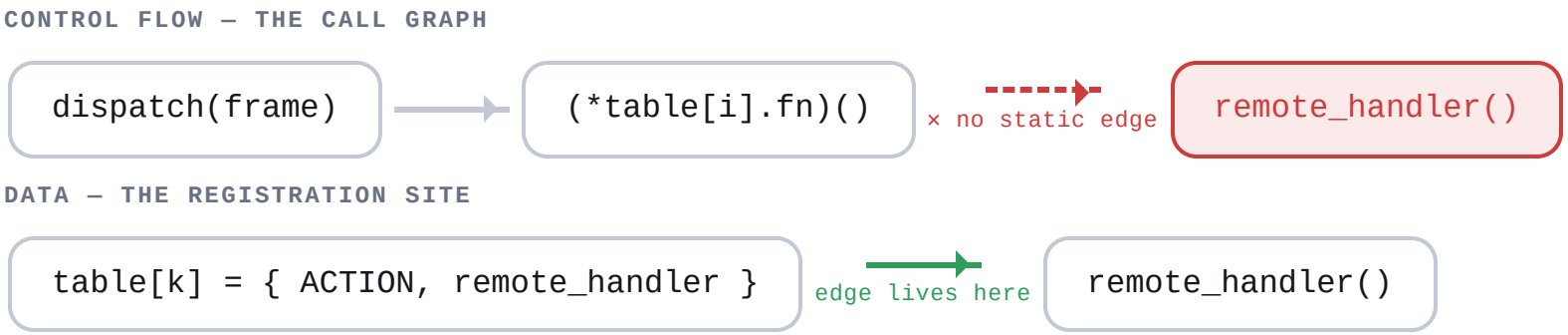


- Security questions are phrased over **protocol actions**, not functions
- The handler is where untrusted payload first becomes program state
- Without the mapping, every later analysis has **no anchor** to start from

DISPATCH BREAKS THE CALL GRAPH

Dispatch leaves no handler edge in the call graph

Function-pointer dispatch drops the caller→handler edge; the binding survives only in data.

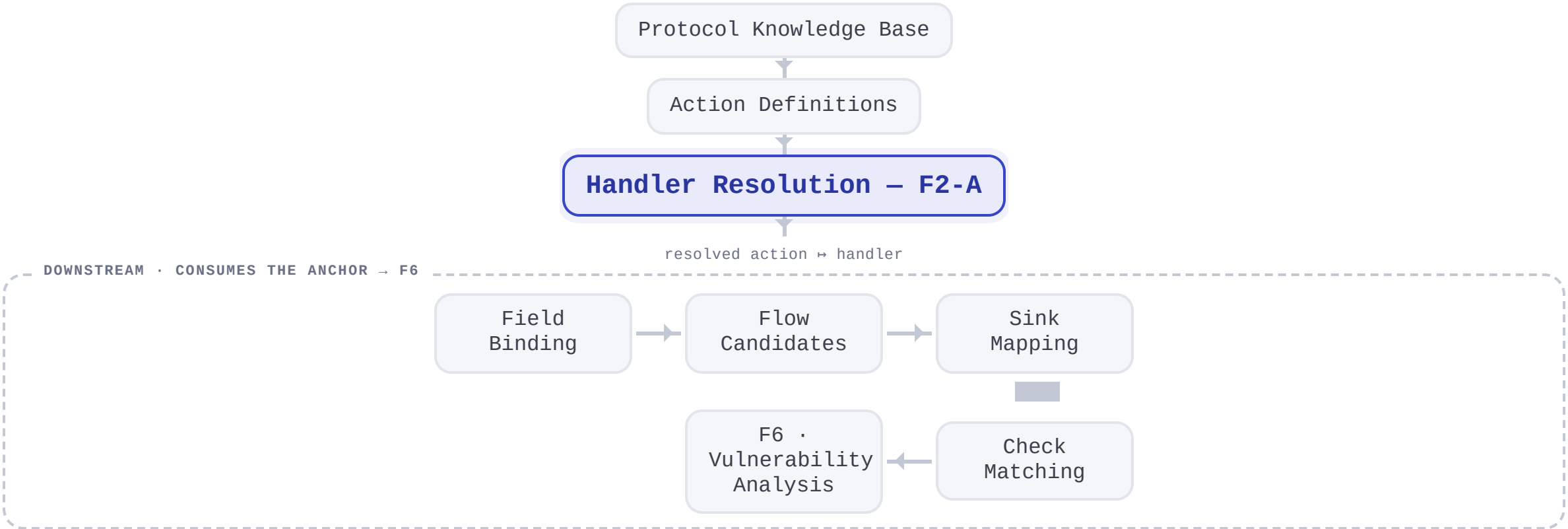


- Dynamic dispatch ⇒ the caller→handler edge is **absent** from the call graph
- The real binding is expressed in **data** (registration sites), not control flow
- Recover it from **registration structure + identifier matching**, not from calls

F2-A IN THE LARGER PIPELINE

F2-A gates every downstream analysis stage

No protocol-aware stage can start until F2-A resolves the action → handler anchor.

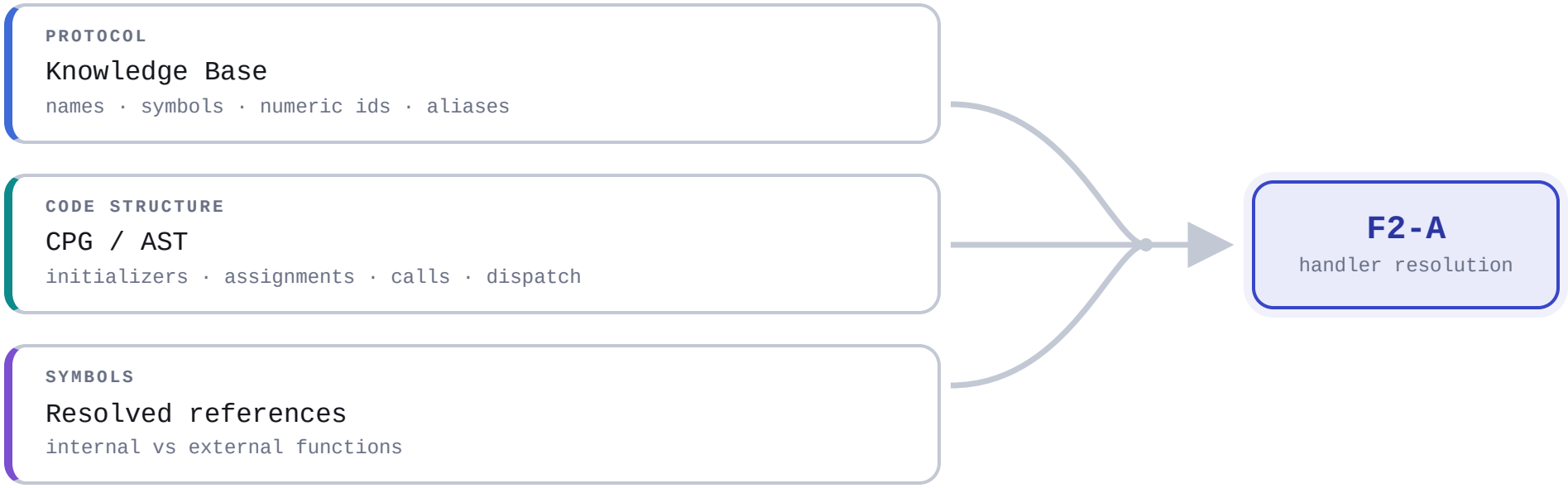


- Upstream: KB + action definitions describe **what to look for**
- F2-A turns actions into **concrete code anchors**
- Downstream consumes the anchor; **F6** makes the final judgment

INPUTS

Resolution fuses protocol truth with code structure

Three independent domains — protocol, structure, symbols — converge into one resolver.

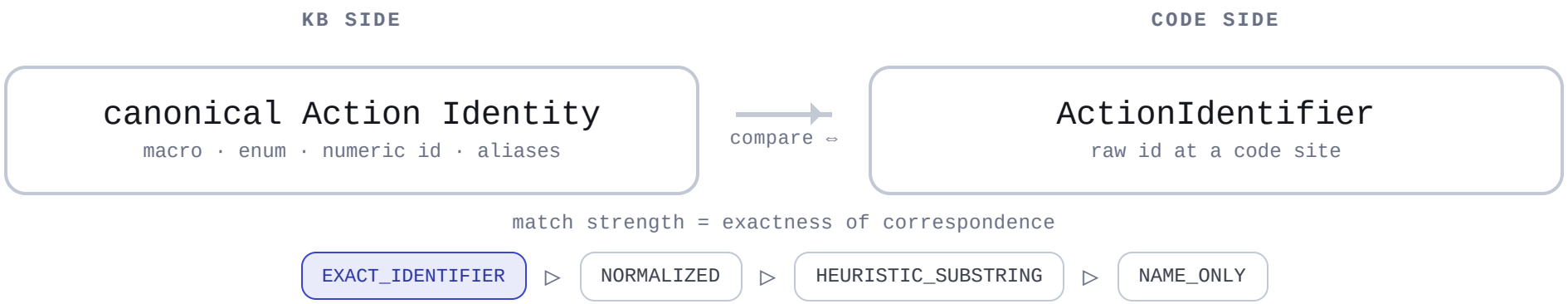


- **Protocol KB** — wire name, macro/enum symbols, numeric ids, aliases
- **Symbols** — which references resolve to internal functions
- **CPG / AST** — shape of initializers, assignments, calls, dispatch sites
- **Action identifiers** — the tokens that must match code ↔ protocol

PROTOCOL IDENTIFIER NORMALIZATION

One action identity unifies many code spellings

The KB folds every spelling of an action into one canonical identity.



- The KB provides the **semantic identity** that unifies many syntactic representations
- Extractors compare a **raw id against the canonical identity**, not strings ad hoc
- The match-strength ladder is **which representation matched, how exactly**

REGISTRATION MECHANISMS: A TAXONOMY

Firmware binds handlers in many different shapes

One semantic — bind action ↦ handler — expressed in many syntactic forms.

AGGREGATE INIT

```
{ ACTION, fn }
```

DESIGNATED INIT

```
{ .action = ACTION, .fn = fn }
```

INDEXED ASSIGN

```
handlers[ACTION] = fn
```

CORRELATED STORES

```
t[k].action = ACTION; t[k].fn = fn
```

REGISTRAR CALL

```
register(ACTION, fn) → callee stores it
```

DELEGATED REGISTRAR

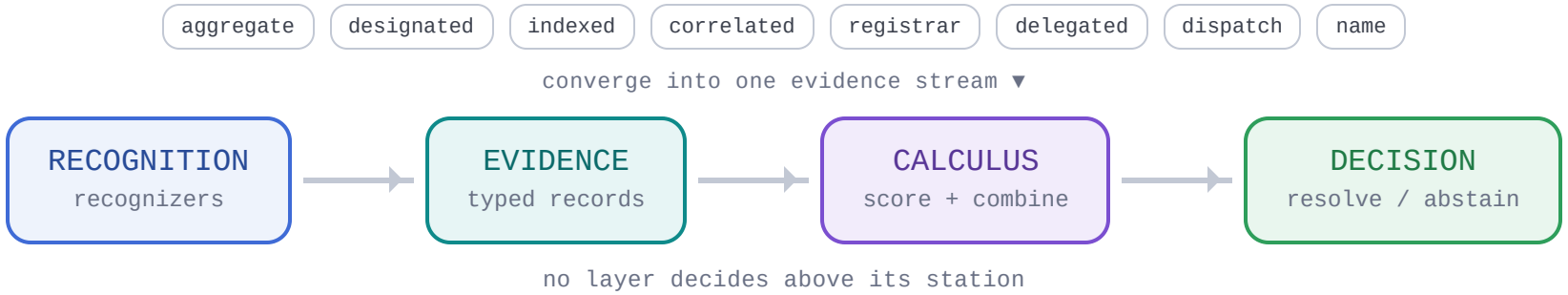
```
register(…) → store_handler(…) → t[.] = fn
```

- Same semantic (**bind action ↦ handler**), many syntactic forms
- Forms differ in **where the id and callback are**, and how they correlate
- Some resolve at initialization; some require **following a call** into a callee

WHY MULTIPLE EXTRACTORS, NOT ONE MATCHER

Extractors recognize; they never decide

A uniform evidence stream separates recognition from every downstream layer.



- Mechanisms correlate id↔callback differently (same node / sibling field / shared slot / arg→param)
- Independent extractors ⇒ independent evidence ⇒ **corroboration** is possible
- A miss in one extractor is a **precise diagnostic**, not a whole-system failure

RECOGNITION LAYER: EXTRACTOR CAPABILITIES

Match strength, not kind, limits a candidate

Extractors emit typed evidence; no kind is reserved out of resolving.

Recognition › Evidence › Calculus › Decision

EXTRACTOR	RECOGNIZES	EMITS	BOUNDARY
REGISTRATION EVIDENCE · EXPLICIT ID ↔ CALLBACK BINDING			
aggregate init	{ ID, fn }	REGISTRATION_INIT	inline id only
designated init	{ .a=ID, .fn=fn }	REGISTRATION_INIT	same struct init
indexed / field	h[ID]=fn ; t[k].*=…	REGISTRATION_ASSIGN	shared slot key
registrar call	f(ID, fn) → stores fn	REGISTRAR_CALL	must reach store
delegated	f→g→ store (arg→param)	REGISTRAR_CALL	depth-bounded
DISPATCH & NAME EVIDENCE · OBSERVED INVOCATION / LEXICAL MATCH			
dispatch site	switch(id) → handler	ENUM_CASE / STRING_DISPATCH	high prior · exact-id capable
name similarity	fn name ≈ action	NAME_MATCH	low-signal fallback

- Per-kind prior is **comparable** across dispatch and registration; name-matching is the weak fallback
- A missing exact-identifier match (the **weak-only cap**) limits a candidate — independent of kind
- Every extractor's boundary is a **named limitation**, surfaced downstream

THE EVIDENCE MODEL

Every extractor speaks one evidence record

A typed record with provenance and an auditable source trail.

Recognition › **Evidence** › Calculus › Decision

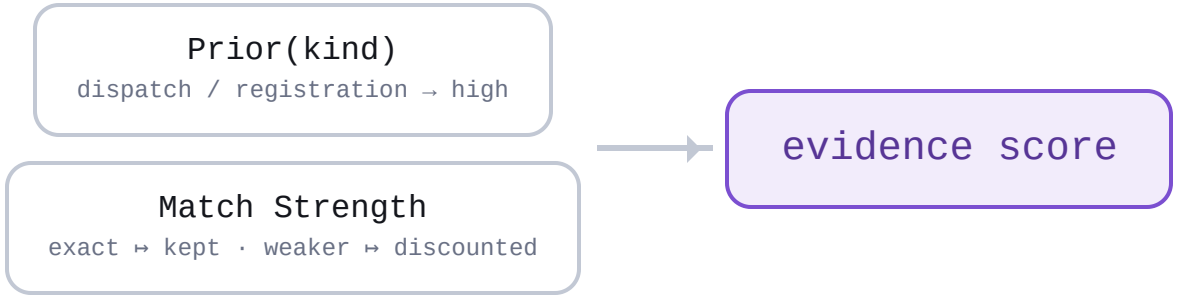
EVIDENCE · record	
action id	matched identifier (+ match strength)
callback	the resolved internal function
kind	which mechanism produced it
provenance	site / group key (dedup + grouping)
confidence	Prior(kind) shaped by match strength
dispatch site	where the binding is observed
records[]	ordered file/line trail (auditable)

- Evidence is **typed by mechanism** and **scored by match quality**
- Provenance key makes duplicates and same-site evidence identifiable
- Records preserve the original **source expressions and locations**

EVIDENCE SCORING

One evidence's score = prior shaped by match

Each evidence scores on its kind's prior, shaped by identifier match strength.

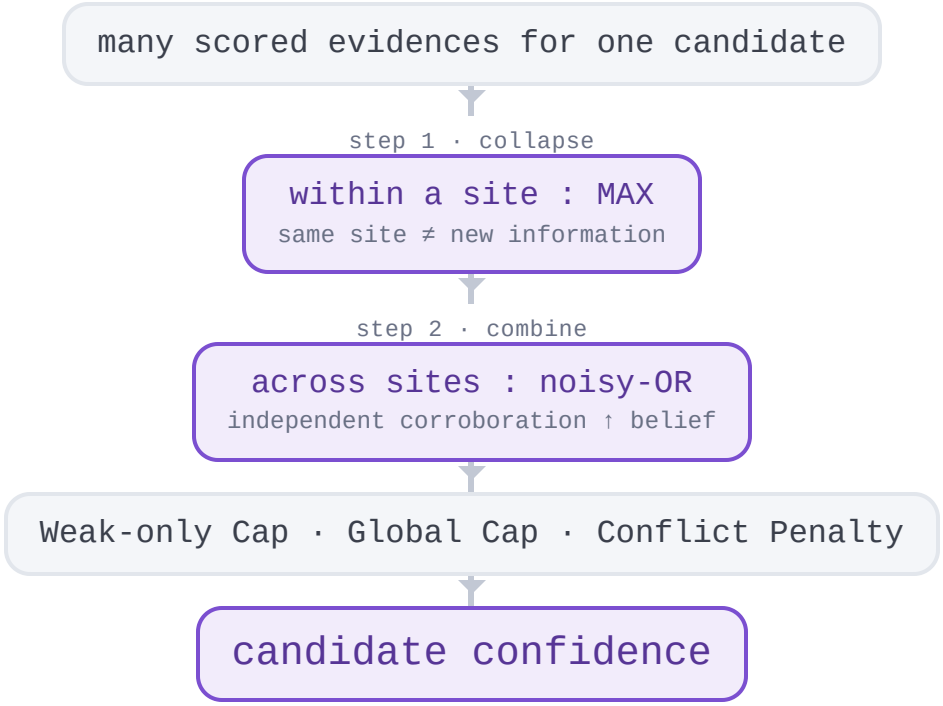


- **Prior(kind)** — dispatch and registration are the high-prior kinds; name-matching the weak fallback
- **Match Strength** — exact identifier preserved; weaker correspondences discounted
- Output: a per-evidence score; **no candidate merging** happens yet

EVIDENCE COMBINATION

Collapse duplicates first, then combine independents

Within-site MAX removes duplicates; cross-site noisy-OR then rewards corroboration.



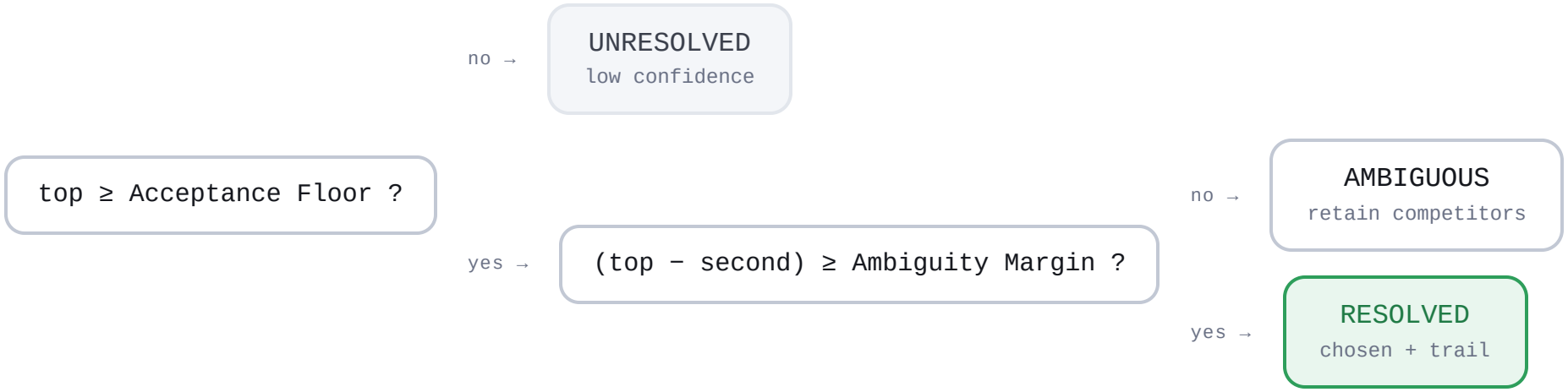
- **Collapse:** same-site duplicates reduce to their MAX — a site can't inflate itself
- **Combine:** only then do independent sites corroborate (noisy-OR)
- Weak-only cap keys on **match strength** (missing exact-id, any kind); competitors penalized

CANDIDATE SELECTION

Resolve, refuse, or abstain — on confidence alone

Acceptance Floor, then Ambiguity Margin, decide — with no structural gate.

Recognition › Evidence › Calculus › **Decision**

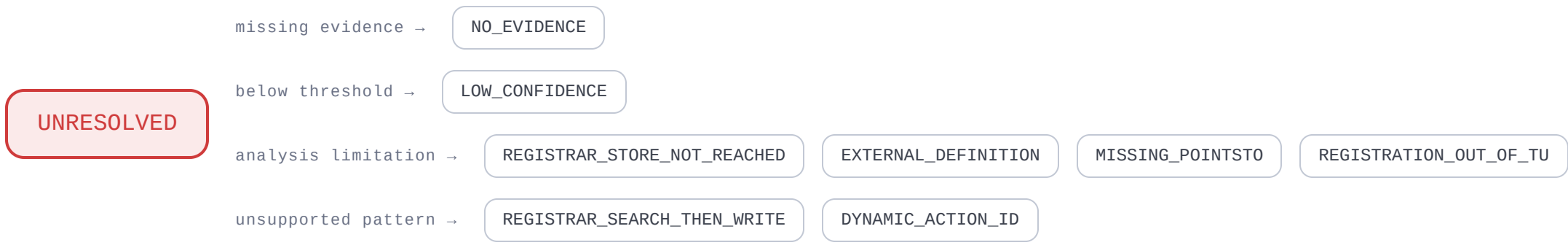


- **RESOLVED:** above the Acceptance Floor and clear of the runner-up
- **AMBIGUOUS:** multiple credible candidates within the Ambiguity Margin → no choice
- **No structural gate:** weak-matched candidates are capped (Slide 12), not barred

DIAGNOSTICS: A TAXONOMY OF “WHY NOT”

Unresolved names the boundary it hit

Every non-resolution is a structured, actionable reason — not a failure.



- Reasons distinguish **“no signal”** from **“signal we intentionally don't follow”**
- Specializations exist (search-then-write ⊂ store-not-reached) for actionability
- These are **scope statements**, not bugs

WORKED EXAMPLE

Match strength picks the winner, not kind

Two extractors fire; the stronger identifier match resolves — here, dispatch.

```
static Reg t[] = { { ACTION_DATA_TRANSFER, foo } };      // registration
switch (f->action){ case ACTION_DATA_TRANSFER: bar(); }  // dispatch
```

RECOGNITION → EVIDENCE

```
A · REGISTRATION_INIT → foo  ·  id=ACTION_DATA_TRANSFER (substring match)
B · ENUM_CASE → bar  ·  id=ACTION_DATA_TRANSFER (normalized match)
```

CALCULUS

bar · 0.72 vs foo · 0.56 → DECISION → RESOLVED → bar

remove the switch → only A → foo resolves alone (≈0.56, just clears the floor)

- Two independent extractors, same action id, **different callbacks** (registration vs dispatch)
- The dispatch evidence matched more strongly → bar clears the margin → **resolves to bar**
- The decision is a function of evidence and **match strength, not of kind**

OUTPUT SCHEMA

Non-resolution is a first-class result

One authoritative per-action result set, every candidate evidence-linked.

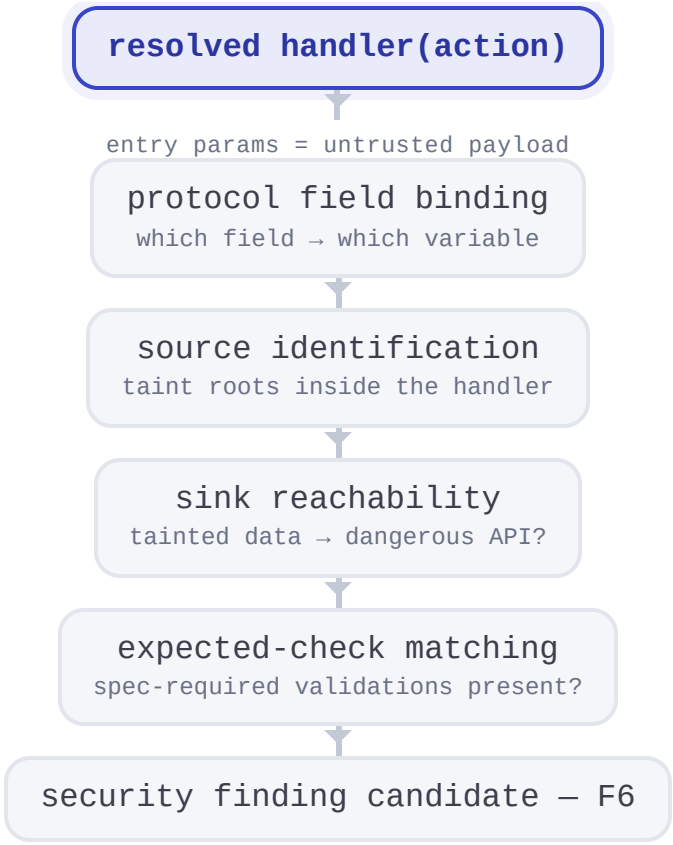
F2-A RESULT	
handler_resolutions	authoritative · one per action
· **status**	RESOLVED / AMBIGUOUS / UNRESOLVED
· **chosen**	present only when RESOLVED
· **candidates**	each carries its evidence + records
· **conflict**	competitors + margin (AMBIGUOUS)
· **unresolved**	reason + attempts (UNRESOLVED)
handler_maps	resolved-only view · back-compat
limitations	global scope caveats

- `handler_resolutions` is the source of truth; `handler_maps` is a resolved-only subset
- Every candidate carries its **evidence and record trail**
- Conflict / unresolved reports make **non-resolution first-class**

DOWNSTREAM CONSUMPTION

The resolved handler anchors all later analysis

Field binding, sources, sinks, and checks all expand from the handler.



- Handler = the function where payload **enters program semantics**
- Field binding and sink search are **scoped by the handler**
- A missing/ambiguous resolution ⇒ downstream **narrows scope or defers**

CURRENT SCOPE

The baseline resolves what structure decides

Statically-decidable registration and dispatch forms, with explicit ambiguity handling.

- ✓ aggregate initialization
- ✓ indexed / correlated field store
- ✓ delegated registrar (bounded)
- ✓ enum/switch dispatch recognition
- ✓ ambiguity detection & abstention
- ✓ designated initialization
- ✓ registrar tracing (reaches store)
- ✓ symbolic receiver correlation (shared slot)
- ✓ name-match fallback
- ✓ structured diagnostics

- Everything here is **AST + symbol resolution + bounded call following**
- Ambiguity and non-resolution are supported **outcomes**, not gaps
- Scope defined by **decidability** without heavy semantic reasoning

LIMITATIONS · INTENTIONAL EXCLUSIONS

Value-reasoning patterns are deliberately out of scope

Loop, alias, and cross-procedure cases are detected, named, and deferred.

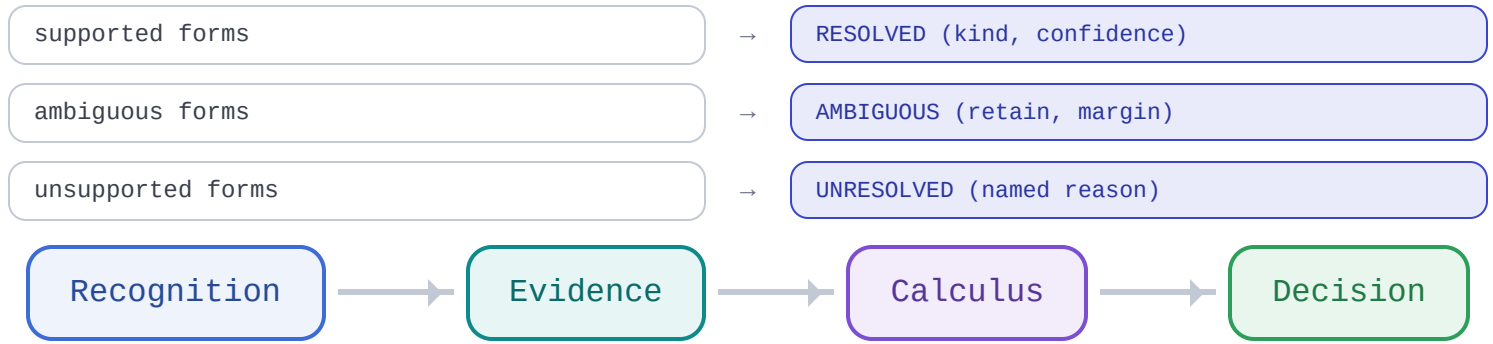
EXCLUDED PATTERN	CAPABILITY IT WOULD NEED
`search-then-write registrar`	`loop + predicate reasoning`
`runtime slot selection`	`symbolic index resolution`
`alias-to-slot binding`	`alias / points-to analysis`
`cross-procedure slot rebuild`	`interprocedural value flow`
`general dynamic action ids`	`DFG-based escalation`

- Each is **detected and named**, not silently mishandled
- Excluded because they trade **precision/soundness** for coverage
- They form a concrete, **prioritizable escalation backlog**

EVALUATION & CONCLUSIONS

Separating recognition from decision is the contribution

Capability and regression tests pin behavior; the backbone makes it extensible.



“By separating recognition from decision through an evidence abstraction, F2-A becomes conservative, auditable, and naturally extensible.”

- Each registration mechanism has a fixture asserting its **exact expected outcome**
- Negative & unsupported cases assert the **right refusal**, not just “no crash”
- The backbone — **Recognition → Evidence → Calculus → Decision** — is the contribution

APPENDIX · NOT PRESENTED

Reference material held for questions

Concrete parameters and mechanics, kept off the main narrative.

HOLD-SLIDES	
A1 · parameters	priors per kind, match-strength multipliers, weak-only cap (match-strength triggered), global cap, acceptance floor, ambiguity margin, conflict penalty, registrar depth — the sole home for numbers
A2 · provenance	why within-group MAX and cross-group noisy-OR; the double-counting failure it prevents
A3 · designated AST	the lowering difference (wrapped member assignments) that makes designated distinct from positional

- Numeric values live **only** here — the main deck stays symbolic
- Match-strength ladder & normalization are **core** (Slide 6), not appendix
- Pull these up only if the room asks for the mechanics